Throws

`std::bad_alloc` if the required internal storage can't be allocated, otherwise `std::future_error` when the effects can't be achieved, or any exception thrown during the construction of `fff` or `xyz...`.

D.5. <mutex> header

The `<mutex>` header provides facilities for ensuring mutual exclusion: mutex types, lock types and functions, and a mechanism for ensuring an operation is performed exactly once.

Header contents

```
namespace std
{
    class mutex;
    class recursive_mutex;
    class timed_mutex;
    class recursive_timed_mutex;
    struct adopt_lock_t;
    struct defer_lock_t;
    struct try_to_lock_t;

    constexpr adopt_lock_t adopt_lock{};
    constexpr defer_lock_t defer_lock{};
    constexpr try_to_lock_t try_to_lock{};

    template<typename LockableType>
    class lock_guard;

    template<typename LockableType>
    class unique_lock;

    template<typename LockableType1,typename... LockableType2>
    void lock(LockableType1& m1,LockableType2& m2...);

    template<typename LockableType1,typename... LockableType2>
    int try_lock(LockableType1& m1,LockableType2& m2...);

    struct once_flag;

    template<typename Callable,typename... Args>
```